# Problem Statement

## One-paragraph statement

LLM-powered software agents increasingly execute model-generated code as part of their tool-use loop, but most deployments rely on isolation designed for trusted multi-tenant workloads rather than for adversarial or malfunctioning code generated by a language model. Naive isolation — a plain Docker container, an unsandboxed subprocess, an in-process `exec` call — leaves agents able to exhaust host resources, persist state across sessions, enumerate the host environment, or, under prompt injection, exfiltrate data through unintended channels. Closed-source services such as OpenAI's Code Interpreter and Anthropic's analysis tool address these risks behind opaque infrastructure; the open-source landscape consists of either heavyweight platforms unsuitable for solo deployment or lightweight wrappers with weak isolation guarantees and no documented threat model. This project builds a hardened, threat-modeled Python execution sandbox for LLM-agent tool-use, with progressively stronger isolation back-ends — hardened Docker, then gVisor — exposed through a clean execution API. Its central contribution is an adversarial evaluation suite that measures isolation strength empirically rather than asserting it, yielding a reusable methodology and an open benchmark for comparing the security properties of agent code-execution sandboxes.

## Why this is a real CS problem (rubric: project selection & problem fit)

- **Applied need.** Agentic systems are now in production at multiple organizations (Anthropic Claude Code, OpenAI Code Interpreter, Cursor, Devin, Aider, plus an expanding open-source ecosystem). Each of them must answer the question "where does this code actually run?" The standard answer in many open-source agents today is "inside whatever process your agent runs in" or "inside a `docker run` with default settings" — neither of which is a defensible answer to a security review.
- **Research-adjacent need.** The 2024–2025 wave of prompt-injection and agent-misuse literature (NIST AI 100-2 2025 update; OWASP LLM Top 10) repeatedly identifies tool-execution isolation as an under-specified risk category. A reproducible benchmark suite for isolation strength would close a measurement gap.
- **Tractable for a 14-week solo project.** Unlike training a model or producing a novel ML technique, the problem decomposes cleanly into (a) building a service, (b) hardening it, (c) measuring it. Each component has well-understood prior art and a clear "done" condition.

## Stakeholders affected by the problem

- **LLM-agent developers** who currently improvise their own isolation and have no benchmark to compare it against.
- **Security engineers** evaluating agent platforms for enterprise adoption who need a vocabulary and evidence for what "safe code execution" means.
- **Researchers** working on prompt injection, agent safety, and tool-use evaluation, who currently lack a shared adversarial test corpus for code-execution channels.
- **End users of agentic products**, who are indirect stakeholders affected by whatever choices their tooling vendors make.

## What success would mean for this stakeholder set

- A reference open-source sandbox implementation that an LLM-agent developer can adopt without writing the isolation layer themselves.
- An empirical comparison of isolation strength vs. performance overhead across two production-realistic back-ends, with confidence intervals.
- A small but well-curated adversarial benchmark (≥40 programs) that researchers and platform teams can extend, fork, or reuse.
- A documented threat model that names what the sandbox does and does not defend against, so downstream users can integrate it responsibly.

## What this project explicitly does **not** claim to solve

- Defense against nation-state-grade attackers, side-channel timing attacks, or rowhammer-style hardware attacks.
- Multi-tenant security where mutually distrusting users share one sandbox instance.
- Network-level isolation for agents that legitimately need outbound network access.
- Sandboxing of GPU-accelerated workloads.
- Replacement for trained-model alignment or input-validation defenses — sandboxing is the last line, not the only line.

## Empirical claims that require citation before submission

The following claims appear in the one-paragraph statement above and **must** be backed by at least one credible source each before W1 submission. The student will verify these as part of the annotated bibliography work.

1. That LLM-agent products such as Code Interpreter and Anthropic's analysis tool exist and execute model-generated code — easy to cite from vendor docs.
2. That open-source agents commonly execute code with weak isolation (Aider, Open Interpreter, AutoGen examples) — citable from project READMEs or recent surveys.
3. That tool-execution isolation appears as an under-specified risk in recent threat-model literature (NIST AI 100-2; OWASP LLM Top 10) — citable from those documents directly.
4. That existing open-source sandbox projects (E2B, Pyodide, Modal sandboxes, Daytona) occupy specific points on the isolation/usability tradeoff — citable from each project's docs.

The bibliography draft in `annotated-bibliography.md` is the place to capture these.

---